Politechnika Lubelska
Wydział Elektrotechniki i Informatyki
Katedra Elektroniki i Technik Informacyjnych

Systemy wizyjne

Prowadzący laboratorium: mgr inż. Marek Kida

Ćwiczenie laboratoryjne nr 6

Temat: Klasyfikacja obiektów za pomocą alg. YOLO

Piotr Bednarski

Lublin, 22.12.2025 r.

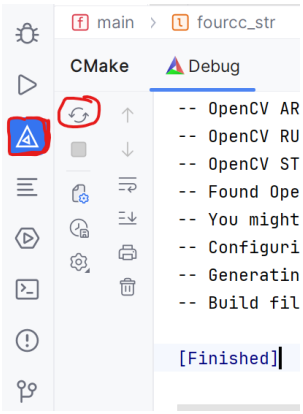
Ćw. 1. Detekcja obiektów

Utwórz strukturę folderów, w których znajdować się będą pliki z poniższego ćwiczenia laboratoryjnego. Struktura ta ma wyglądać następująco:

```
/home/student/Documents/
    - number_albumu/
        - L6/
            - latex/
            - code/
```

Do folderu `latex` wklej rozpakowany szablon sprawozdania.

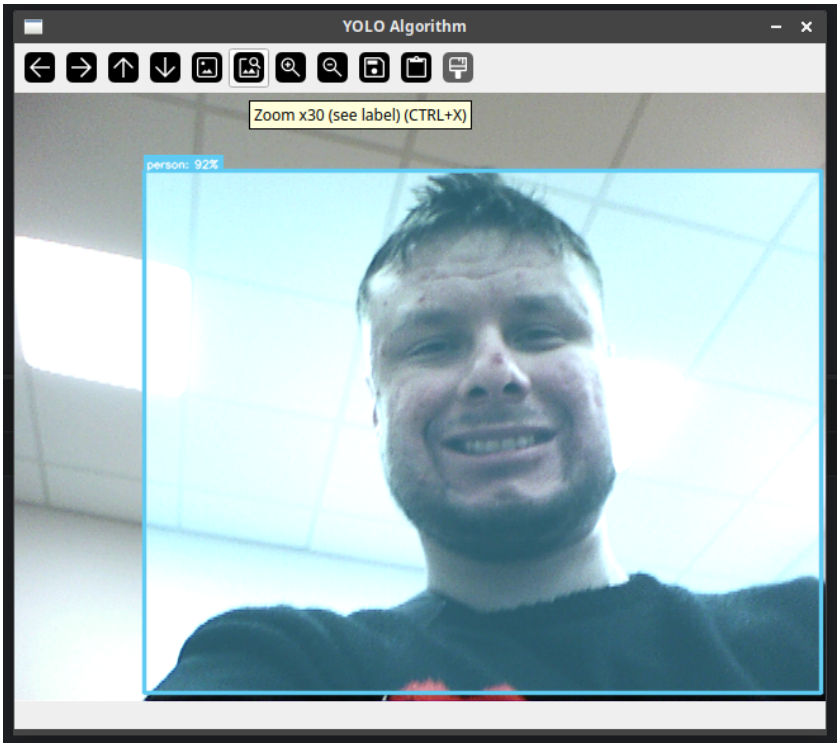
Do folderu `code` wstaw projekt z platformy Moodle. Otwórz ten folder z projektem w programie CLion (*File -> Open...*). Upewnij się, że otwarty jest tylko ten projekt. Odśwież zawartość projektu CMake w panelu na lewym dolnym roku ekranu. Dzięku temu edytor C++ będzie znajdował bibliotekę OpenCV i YOLO-CPP.



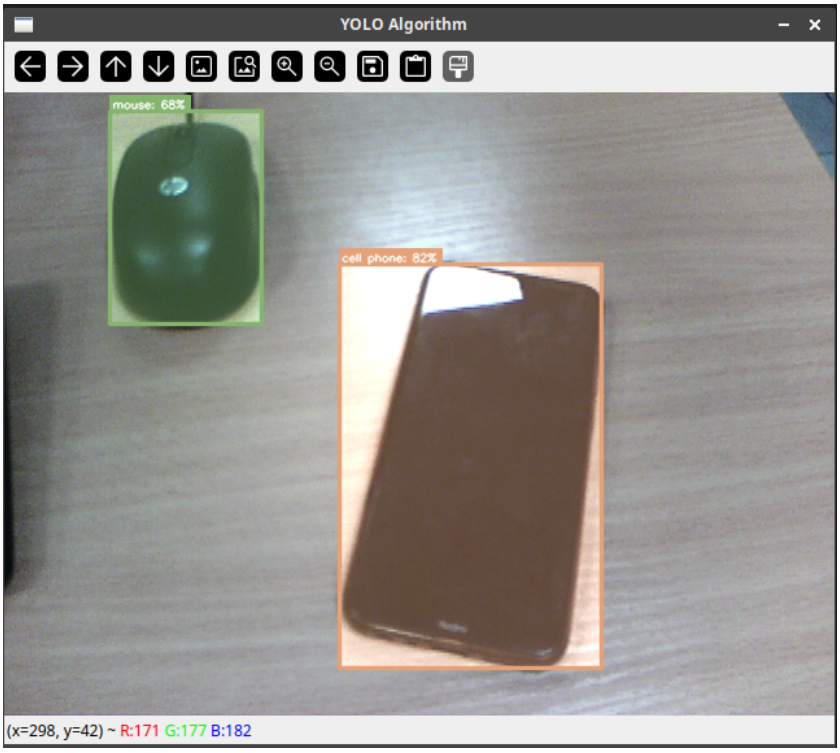
Rysunek 1: Przeładowanie projektu CMake

Wykonaj następujące czynności

1. Uchruchom plik wykonywalny `main` (górna część ekranu) i sprawdź co widać na obrazie. Zrób kilka zdjęć z obiektami jakie zostają wykryte. Ich lista jest skończona: zawarte one są w pliku `models/coco.names`. Wyświetl kilka obiektów z tej listy na ekranie i ustaw kamerę tak, aby je wykryć.



Rysunek 2: Wykryty obiekt klasy `person`



Rysunek 3: Wykryte obiekty klas `mouse` i `cell phone`

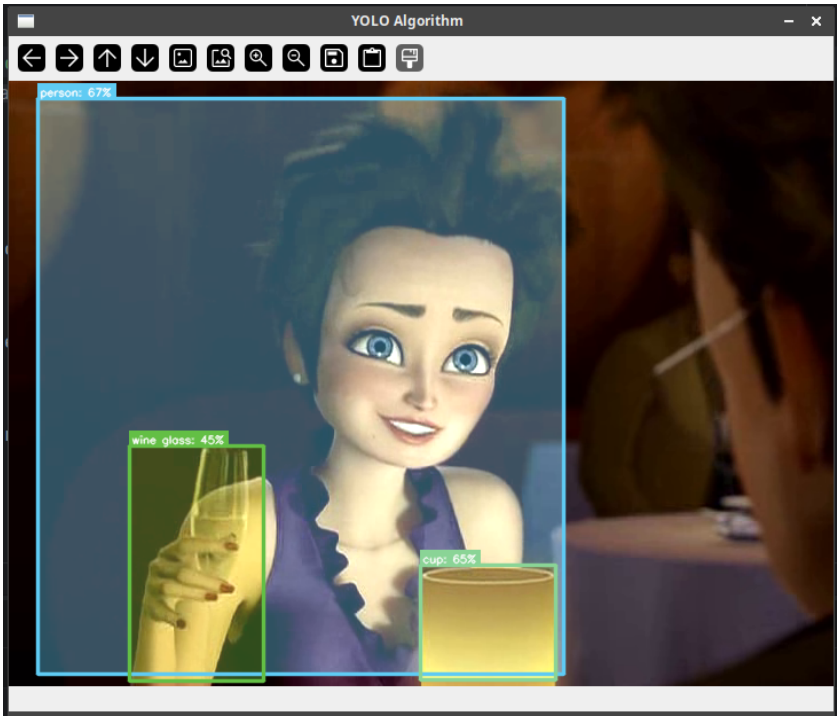
2. Jak dużo czasu średnio zajmuje każdy etap przetwarzania danych? Odczytaj z terminala, oblicz sumaryczny czas i przelicz ile to mniej więcej klatek na sekundę.

```
Overall detection took 70.9801 milliseconds.
preprocessing took 3.08794 milliseconds.
postprocessing took 2.83797 milliseconds.
```

Rysunek 4: Czasy przetwarzania pojedyńczej klatki

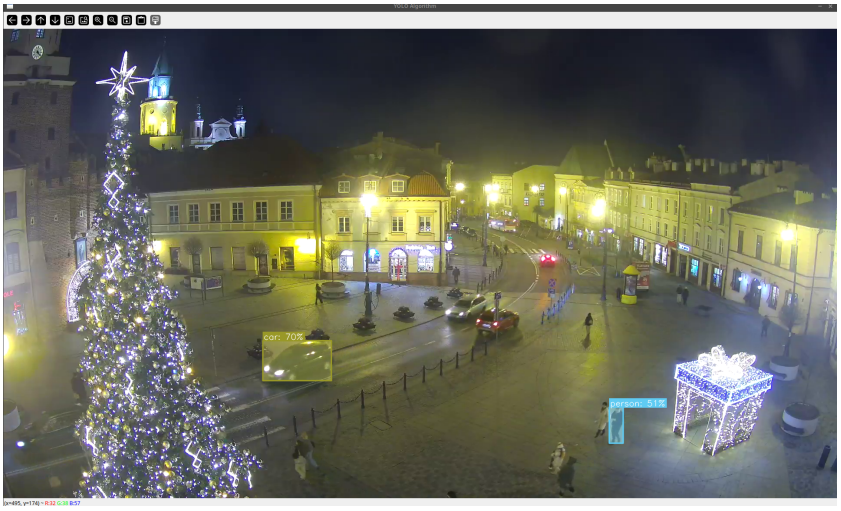
Sumarycznie daje to `76.90601 ms`, co w przeliczeniu daje około `13.003 fps`.

3. Sprawdź jak algorytm sprawdza się na strumieniu wideo (np. kamery Lublin) oraz w pliku `data/megaminds.avi`.



Rysunek 5: Algorytm YOLO klasyfikujący obiekty w `megaminds.avi`

Jak widać na rysunku 5 algorytm sprawuje się dobrze. Obiekty są poprawnie klasyfikowane i w dodatku nie mniej płynnie niż na obrazie z kamery. Na zdjęciu widoczna postać ludzka klasy `person` oraz kieliszek wina klasy `wine glass`. Niepoprawnie sklasyfikowana została lampka jako klasa `cup`.



Rysunek 6: Algorytm YOLO klasyfikujący obiekty z obrazu na żywo z bramy krakowskiej

Jak widać na rysunku 6 algorytm radzi sobie w tym przypadku gorzej - większość samochodów na obrazie nie jest wogóle klasyfikowana, to samo tyczy się ludzi - z nich tylko jedna osoba została poprawnie sklasyfikowana. Algorytm działa również wyraźnie wolniej, co przedstawia rysunek 7.

```
Overall detection took 130.213 milliseconds.
preprocessing took 3.78304 milliseconds.
postprocessing took 1.68848 milliseconds.
Overall detection took 107.392 milliseconds.
preprocessing took 1.70497 milliseconds.
postprocessing took 1.69642 milliseconds.
Overall detection took 123.34 milliseconds.
preprocessing took 1.24901 milliseconds.
postprocessing took 1.68073 milliseconds.
```

Rysunek 7: Wolniej działający algorytm na obrazie o większej rozdzielczości

4. Sprawdź też jak algorytm sprawuje się na pojedynczym obrazie `data/bus` (pominąć wtedy trzeba linie z `VideoCapture` oraz pętlę `while`, natomiast odczytać trzeba obraz za pomocą `cv::imread()`);



Rysunek 8: Klasyfikacja na obrazie `bus.jpg`

Jak widać tutaj klasyfikacja jest niemal poprawna - prawidłowo wykryto bus oraz 3 osoby. Jednakże na pytanie: ile osób widocznych jest na obrazie moim zdaniem poprawną odpowiedzią jest 4 - po lewej stronie obrazu widać rękę oraz fragment odzieży czwartej osoby.

5. Które obiekty są łatwe do wykrycia, a z którymi algorytm miewa problemy?

Łatwiejsi dla algorytmu YOLO do wykrycia wydają się ludzie, jednakże algorytm ten miewa problemy z wykrywaniem obiektów oddalonych, zajmujących niewielką część obrazu, a także takich, których tylko fragment jest widoczny na obrazie. Ogólnie jeśli przedmiot zajmuje znaczną część przetwarzanego obrazu - prawdopodobnie zostanie sklasyfikowany prawidłowo.

Ćw. 2. Segmentacja obiektów

Zmodyfikuj kod programu.

1. Wejdź do pliku README.md w głównym katalogu projektu. Znajdują się tam przykłady jak użyć kolejnych funkcjonalności bibliotek algorytmu YOLO. Podmienić trzeba na segmentację:

   a) dyrektywa `#include` (pliki nagłówkowe znajdują się w folderze `include`.

   b) ścieżka modelu sieci neuronowych na taką obejmującą nową funkcjonalność,

   c) detektor YOLO,

   d) wektor detekcji (na wektor segmentacji),

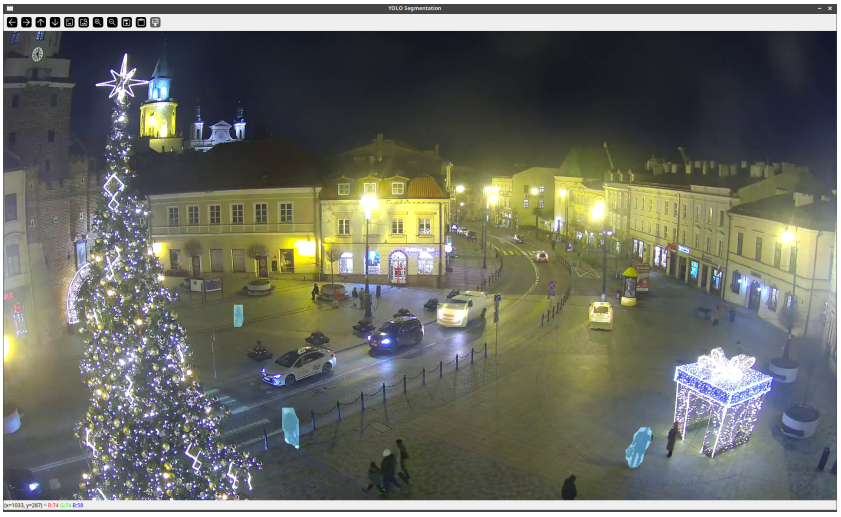
   e) rysowanie wyniku na pojedynczej klatce.

   Dobieraj najnowsze algorytmy dla kolejnych zastosowań (YOLOv11).

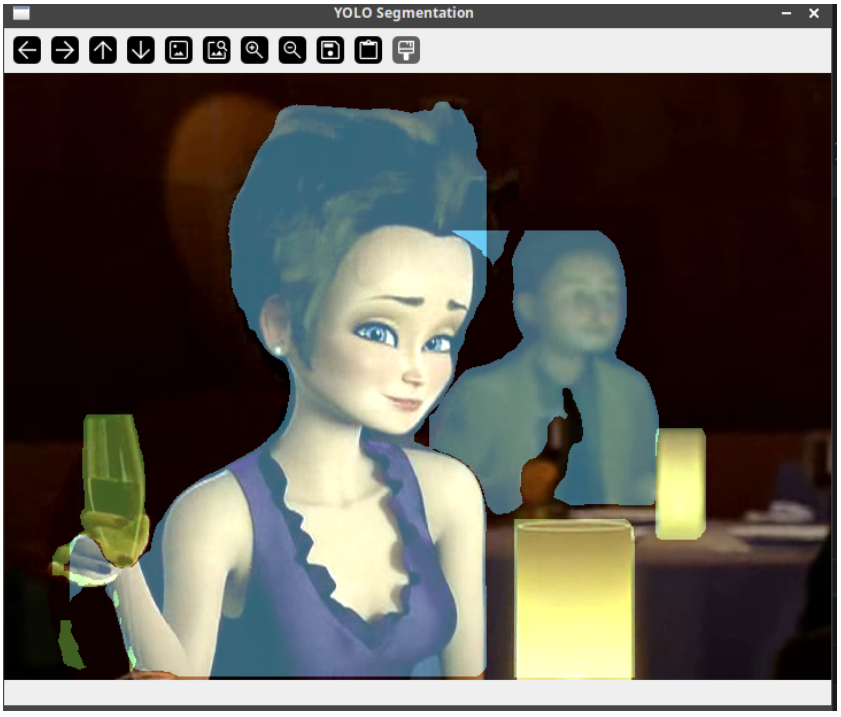
2. Sprawdź wyniki na kamerze, źródle online pliku wideo i statycznym obrazie (jedno zdjęcie do każdego).



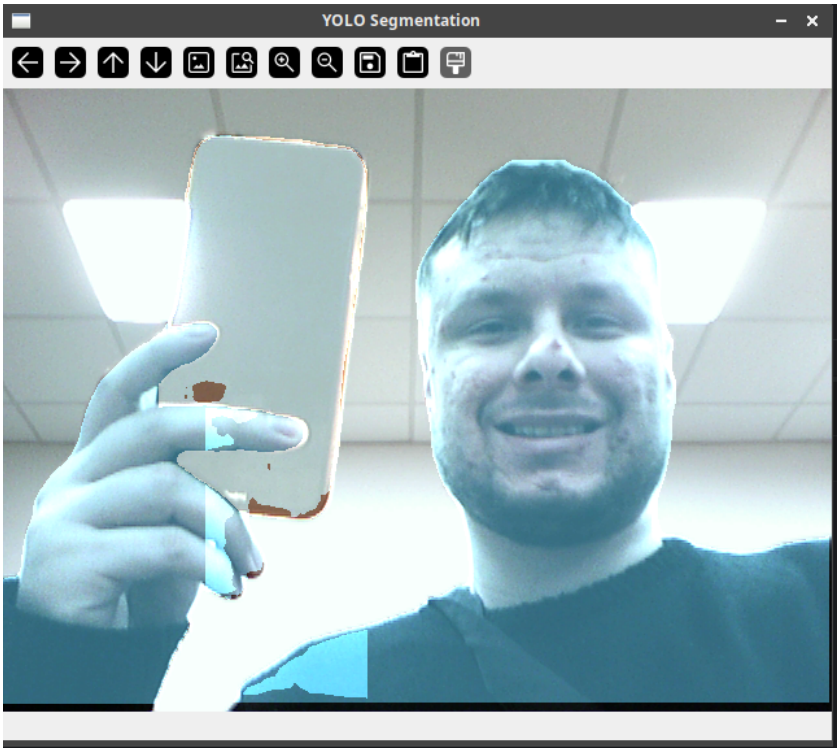
Rysunek 9: W roli źródła statyczny obraz `bus.jpg`



Rysunek 10: W roli źródła obraz na żywo z Bramy Krakowskiej



Rysunek 11: W roli źródła plik wideo `megamind.avi`



Rysunek 12: W roli źródła obraz z kamery

3. Napisz czym różni się segmentacja or detekcji i kiedy jedno jest lepsze od drugiego.

   Segmentacja różni się od detekcji tym, że segmentacja to zaznaczenie dokładnego obszaru, który algorytm uznaje za przynależny do określonej klasy, natomiast detekcja to bardziej ogólne stwierdzenie przynależności obiektu do danej klasy, a następnie narysowanie prostokąta o podobnej wielkości do klasyfikowanego obiektu mniej więcej na jego pozycji.

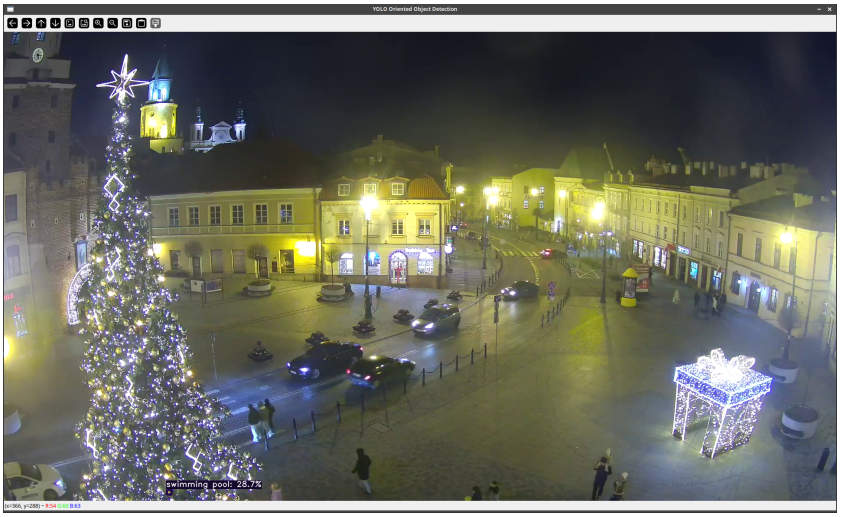
   Segmentacja jest lepsza, gdy z punktu widzenia systemu istotne jest dokładne rozpoznanie kształtu obiektu.

   Detekcja jest lepsza, gdy z punktu widzenia systemu istotna jest szybkość działania algorytmu - algorytm segmentacji działa wyraźnie wolniej.

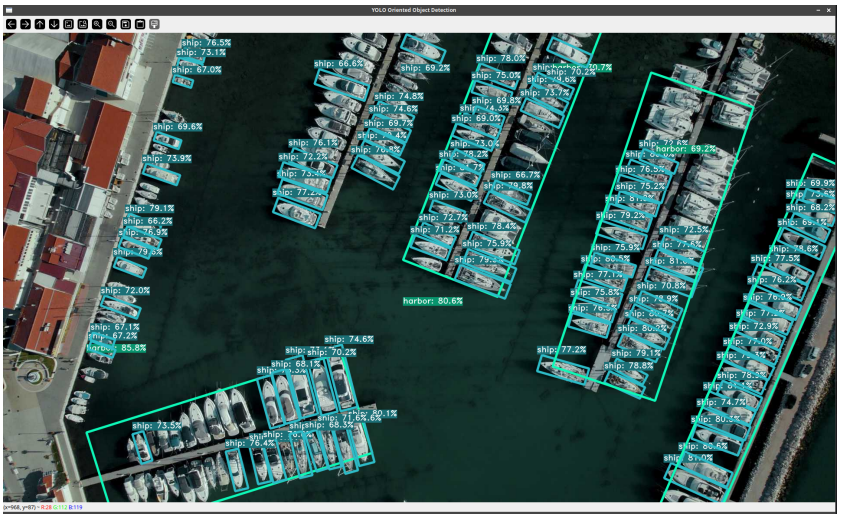
Ćw. 4. Zorientowane otaczanie wykrytych obiektów – OBB

Zmodyfikuj kod programu analogicznie do poprzedniego zadania.

1. Zmień model sieci neuronowej na `yolo11n-obb.onnx`
2. Sprawdź działanie algorytmu YOLO-OBB na obrazie `data/boats.jpg` oraz na strumieniu z Bramy Krakowskiej.



Rysunek 13: Działanie algorytmu YOLO OBB na strumieniu z Bramy Krakowskiej



Rysunek 14: Działanie algorytmu YOLO OBB na obrazie `boats.jpg`

3. Porównaj działanie tego algorytmu i zwykłej detekcji obiektów.

   Działanie algorytmu OBB różni się tym od zwykłej detekcji, że rysowane prostokąty przez algorytm OBB są obrócone w takiej orientacji, w jakiej jest wykrywany obiekt. Zwykła detekcja natomiast pozwala na zaznaczenie wykrywanego obiektu jedynie nieobróconym prostokątem - to znaczy takim, którego szerokość jest równoległa do szerokości monitora, a wysokość - do jego wysokości.

4. Zapisz wnioski z działania tego algorytmu.

   Algorytm OBB działa wolniej niż zwykła detekcja. Nie nadaje się również do przetwarzania obrazu z Bramy Krakowskiej - prawdopodobnie sieć neuronowa algorytmu YOLO OBB trenowana była tylko i wyłącznie za pomocą etykiet klas dotyczących portowych i morskich obiektów.

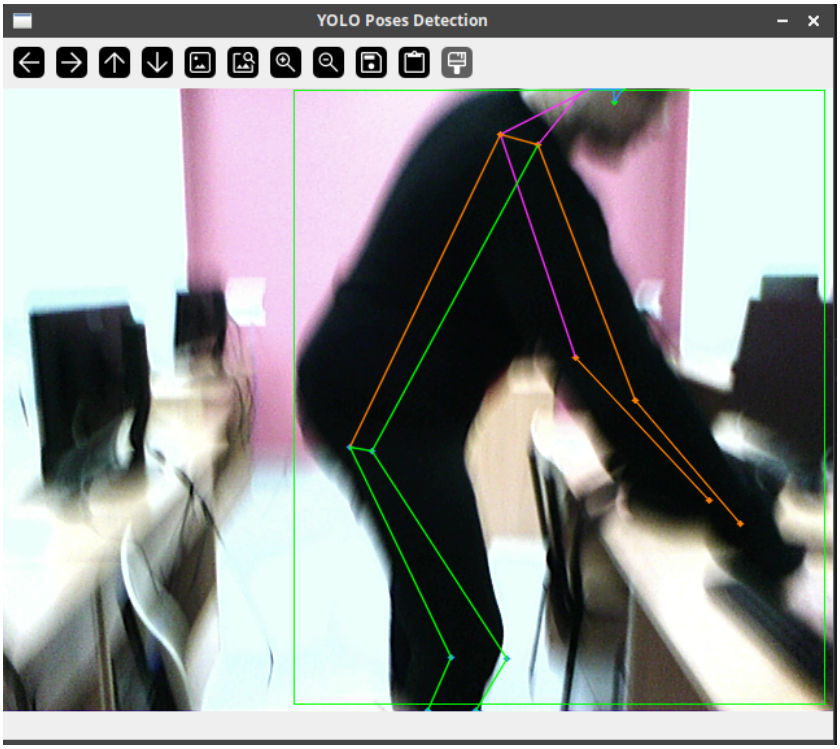
Ćw. 4. Wyznaczanie pozycji ciała

Zmodyfikuj kod programu analogicznie do poprzedniego zadania.

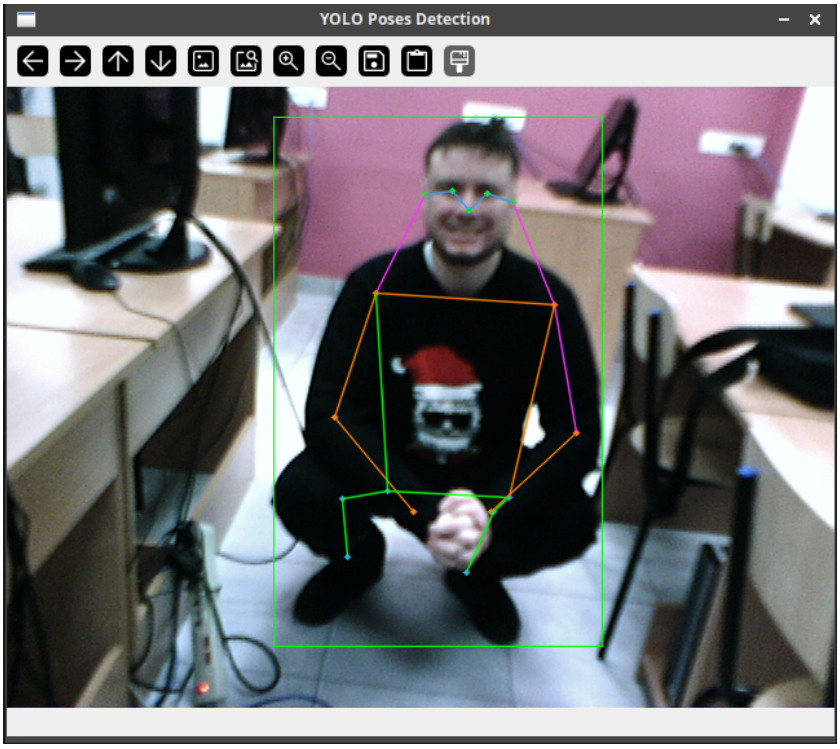
1. Zmień model sieci neuronowej na `yolo11n-pose.onnx`
2. Sprawdź w kilku pozycjach działanie algorytmu na kamerze. Sprawdź też działanie na pliku `data/bus.jpg`



Rysunek 15: Efekt działania dla obrazu `bus.jpg`



Rysunek 16: Efekt działania dla obrazu z kamery - pozycja stojąca



Rysunek 17: Efekt działania dla obrazu z kamery - kucnięcie

3. Zapisz wnioski z działania tego algorytmu.

   Algorytm YOLO do wyznaczania pozycji ciała działa zadowalająco. Pozycja została poprawnie wykryta zarówno dla rozmazanego obrazu z rysunku 16, jak i ostrego z rysunku 17. Pozycje zostały również prawidłowo wykryte dla obrazu `bus.jpg`. Algorytm jest również szybki w porównaniu do OBB czy Segmentacji, co przedstawia rysunek 18.

```
Overall detection took 45.6295 milliseconds.
preprocessing took 2.98743 milliseconds.
postprocessing took 0.021637 milliseconds.
Overall detection took 64.4247 milliseconds.
preprocessing took 2.63097 milliseconds.
postprocessing took 0.021912 milliseconds.
Overall detection took 46.5411 milliseconds.
preprocessing took 2.67803 milliseconds.
postprocessing took 0.021894 milliseconds.
```

Rysunek 18: Czas przetwarzania kilku klatek

Zakończenie

Upewnij się, że strona tytułowa jest uzupełniona. Prześlij wygenerowany plik `.pdf` jako zadanie na platformę Moodle (aktualny plik `.pdf` tworzony jest w momemcie kompilacji w programie TexStudio). Zamknij plik w programie TexStudio i projekt w programie CLion (*File -> Close project*).